

Neighbour: A Full-Stack Web-Based Local Services Marketplace for Khulna City

by

CSE 4100

Field Work / Industrial Training



Department of Computer Science and Engineering
Northern University of Business and Technology Khulna
Khulna-9208, Bangladesh

August 2026

Declaration

This is to certify that the Field Work / Industrial Training work entitled "Neighbour: A Full-Stack Web-Based Local Services Marketplace for Khulna City" has been carried out by Sabrina Ibrahim in the Department of Computer Science and Engineering, Northern University of Business and Technology Khulna, Khulna-9208, Bangladesh. The above Field Work / Industrial Training work or any part of this work has not been submitted anywhere for the award of any degree or diploma.

Acknowledgement

First and foremost, I would like to express my heartiest gratitude to the Almighty Allah for allowing me to accomplish this Field Work / Industrial Training successfully and for granting me the strength, patience and good health throughout the entire period of my training.

I would like to express my deepest respect and sincere gratitude to my supervisor, Md. Rabiul Islam, Advisory Dean, Department of Computer Science and Engineering, Northern University of Business and Technology Khulna, for the continuous guidance, insightful suggestions and constant encouragement that shaped this work from the first requirement discussion to the final deployment of the system.

I am also grateful to the management and the engineering team of Appstick, Khulna, where the industrial training was conducted, for providing me a professional working environment, real project exposure and generous mentorship during the training period. Their feedback on software engineering practices, code quality and teamwork has been invaluable to my professional growth.

I extend my thanks to all the respected faculty members and staff of the Department of Computer Science and Engineering, NUBTK, for building the academic foundation that made this training possible. Finally, I am forever indebted to my parents and family members for their unconditional support, patience and prayers.

Contents

	Page
Title Page	i
Declaration.....	ii
Acknowledgement	iii
Contents	iv
List of Tables	vi
List of Figures.....	vii
List of Equations	viii
List of Algorithms	ix
List of Abbreviations	x
CHAPTER I Introduction	1
1.1 Introduction.....	1
1.2 Motivation.....	2
1.3 Objectives and Specific Aims	2
1.4 Organization of the Report.....	3
CHAPTER II Methodology	4
2.1 System Architecture.....	4
2.2 Requirement Analysis.....	5
2.3 Database Design.....	6
2.4 Technology Stack.....	7
2.5 REST API and Security Design	8
2.6 Core Algorithms and Business Logic	9
CHAPTER III Results and Discussion.....	11
3.1 Implemented System and Modules	11
3.2 Testing and Validation	15
3.3 Security and Performance	16
3.4 Discussion.....	17

CHAPTER IV	Conclusions and Recommendations	19
4.1	Conclusions.....	19
4.2	Limitations	19
4.3	Recommendations and Future Work	19
References		21

List of Tables

Table No.	Description	Page
2.2	Core database entities and their purpose	6
2.4	Technology stack of the Neighbour platform	7
2.5	Principal REST API endpoints	8
3.1	Implemented modules by user role	11
3.2	Representative test cases and outcomes	16

List of Figures

Figure No.	Description	Page
2.1	Three-tier system architecture of Neighbour	4
2.2	Use-case diagram of the marketplace	6
2.3	Entity-relationship (ER) diagram	6
3.1	Booking lifecycle (status transitions)	12
3.2	Client-side services directory	12
3.3	Professional profile page	13
3.4	Client My Bookings page	13
3.5	Provider statistics dashboard	14
3.6	Provider service management screen	14
3.7	Provider weekly availability configuration	15
3.8	Administrator dashboard	15
3.9	Administrator user and provider management	15

List of Equations

Equation No.	Description	Page
2.1	Platform revenue from completed bookings	9
2.2	Provider average rating re-computation	10

List of Algorithms

Algorithm No	Description	Page
2.1	Double-booking-safe slot booking	9
2.2	Provider rating recomputation on new review	9

List of Abbreviations

NUBTK	Northern University of Business and Technology Khulna
API	Application Programming Interface
REST	Representational State Transfer
CRUD	Create, Read, Update, Delete
RBAC	Role-Based Access Control
ER / ERD	Entity-Relationship (Diagram)
UI / UX	User Interface / User Experience
SQL	Structured Query Language
PDO	PHP Data Objects
HTTP(S)	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
SPA	Single-Page Application
DBMS	Database Management System
CSV	Comma-Separated Values
CSS	Cascading Style Sheets
TS	TypeScript

CHAPTER I

Introduction

1.1 Introduction

The rapid growth of internet connectivity and smartphone adoption in Bangladesh has fundamentally changed the way people discover and consume everyday services. Yet, for the vast majority of households in a city such as Khulna, hiring a trustworthy plumber, electrician, cleaner, private tutor or photographer still depends on informal word-of-mouth referrals, with no reliable way to verify a professional's identity, compare prices or read the experiences of previous customers. This report documents the design, development and outcome of “Neighbour”, a full-stack web-based local services marketplace built during the Field Work / Industrial Training under the CSE 4100 course, which digitally connects residents of Khulna with verified, skilled local service professionals.

Neighbour is organized around three distinct classes of users. A client can browse services and a searchable directory of professionals, view rich profiles, book a service for a specific date and time slot, track and cancel bookings, leave reviews after a completed job, save favourite professionals and chat with them directly. A service provider registers with a National ID card number, waits for administrative approval and then manages services, weekly availability and incoming bookings from a personal dashboard. An administrator oversees the whole platform - approving providers, managing user accounts, publishing announcements and monitoring live sales and revenue. The platform sustains itself through a transparent fifteen percent (15%) commission charged on every completed service.

Technically, Neighbour is implemented as a three-tier application. The presentation tier is a responsive single-page application written in React 19 and TypeScript and bundled with Vite; the application tier is a stateless REST API written in PHP 8.2 running on Apache; and the data tier is a normalized MySQL 8 relational database. The entire stack is containerized with Docker and Docker Compose so that the database, backend and frontend

can be brought up on any machine with a single command, providing a professional, reproducible engineering environment throughout the training.

1.2 Motivation

The motivation for this project arises directly from a well-known everyday problem in urban Bangladesh: the local service market is highly fragmented and largely informal. When a household appliance breaks down or a family needs a tutor, the search typically begins by asking neighbours or relatives. This informal process suffers from several weaknesses. There is no verification of a professional's identity or skill, which raises safety and trust concerns; pricing is opaque and often negotiated on the spot, leading to inconsistent and sometimes unfair charges; and there is no accumulated reputation, so a customer cannot benefit from the experience of others who hired the same person before.

At the same time, thousands of genuinely skilled workers - plumbers, electricians, cleaners, handymen, photographers and tutors have very limited digital visibility and therefore struggle to reach new customers or grow their income. A structured, trustworthy online marketplace can solve both sides of this problem simultaneously: it gives clients a safe, transparent and convenient way to find and book vetted professionals and it gives professionals a low-cost channel to advertise their services, manage their schedule and build a verifiable reputation. Building such a platform, using an industry-standard technology stack and professional engineering practices, was the central motivation of this industrial training.

1.3 Objectives and Specific Aims

The primary objective of this Field Work / Industrial Training was to design and develop a secure, reliable and user-friendly full-stack marketplace that connects clients with verified local service providers in Khulna.

The specific aims pursued to achieve this objective were as follows:

1. To analyze the requirements of the three principal actors - client, service provider and administrator and translate them into a concrete set of functional and non-functional requirements.
2. To design a normalized relational database capable of representing users, professional profiles, service categories, services, bookings, reviews, availability, favourites, messages and notifications.

3. To implement a stateless REST API in PHP 8.2 that enforces role-based access control and protects against common web vulnerabilities.
4. To build a responsive, accessible single-page frontend in React and TypeScript that consumes the API and degrades gracefully when the backend is unavailable.
5. To implement key business logic including double-booking prevention, automatic rating recomputation and a 15% platform-commission revenue model.
6. To containerize the entire application with Docker so that it can be deployed and evaluated consistently across environments.

1.4 Organization of the Report

The remainder of this report is organized as follows. Chapter II (Methodology) presents the development methodology, the three-tier system architecture, the requirement and use-case analysis, the database design, the technology stack, the REST API and security design and the core algorithms and business logic of the platform. Chapter III (Results and Discussion) describes the implemented system and its modules, the testing and validation carried out, the security and performance characteristics achieved and a broader discussion of the outcomes and challenges. Chapter IV (Conclusions and Recommendations) summarizes the work, states its current limitations and outlines recommendations and directions for future development. The report ends with the list of references cited, in IEEE format.

CHAPTER II

Methodology

This chapter describes the engineering procedure followed to build the Neighbour marketplace. An incremental, iterative development methodology was adopted: the system was decomposed into independent modules (authentication, professionals, services, bookings, reviews, messaging and administration) and each module was designed, implemented, integrated and tested before proceeding to the next. This approach allowed continuous verification against the requirements and produced a working, demonstrable system at every stage of the training.

2.1 System Architecture

Neighbour follows a classic three-tier architecture that cleanly separates presentation, application logic and data storage. The separation improves maintainability, allows the frontend and backend to be developed and scaled independently and keeps all sensitive business rules on the server. Figure 2.1 illustrates the overall architecture.

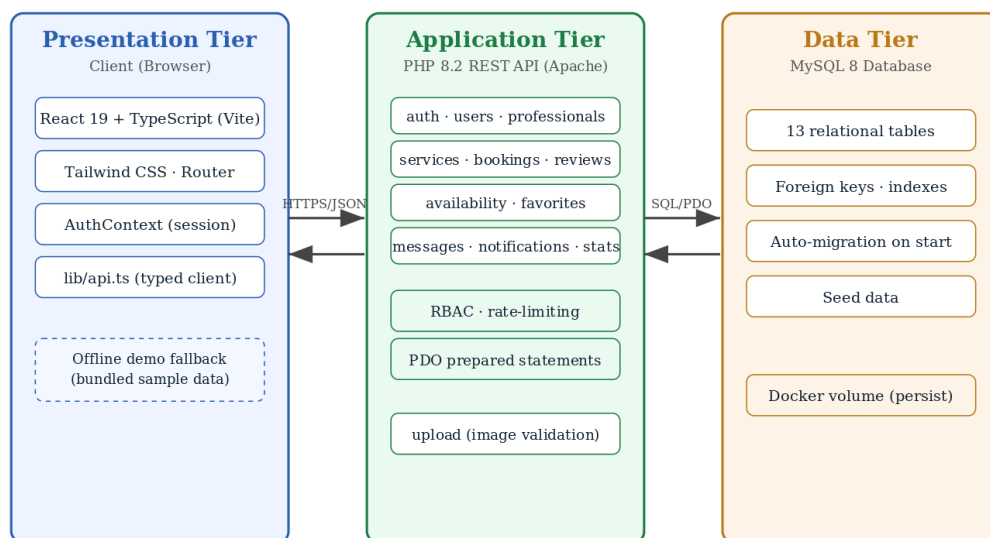


Fig. 2.1: Three-tier system architecture of Neighbour

The presentation tier is a single-page application (SPA) built with React 19, TypeScript and Tailwind CSS and bundled by Vite. It manages authentication state through a React context, communicates with the backend exclusively through a typed API client and includes a graceful offline demo mode that falls back to bundled sample data when the backend is unreachable. The application tier is a PHP 8.2 REST API served by Apache; it exposes resource-oriented endpoints, enforces role-based access control and rate-limiting, and accesses the database exclusively through PDO prepared statements. The data tier is a MySQL 8 database of thirteen normalized tables whose schema is created and self-migrated automatically on first start. All three tiers are packaged as Docker containers and orchestrated by Docker Compose.

2.2 Requirement Analysis

The functional requirements of the system were derived from the goals of its three actors. A client must be able to register and authenticate, browse and search services and professionals, view profiles and reviews, book an available time slot, manage and cancel bookings, leave reviews for completed jobs, maintain a list of favourite professionals and exchange messages. A service provider must be able to register with a National ID, manage a catalogue of services with images, define weekly availability, accept and progress bookings through their lifecycle, and edit their public profile. An administrator must be able to approve or reject providers, activate or deactivate accounts, publish a site-wide announcement, view live sales and revenue statistics and export reports. These interactions are summarized in the use-case diagram of Figure 2.2.



Fig. 2.2: Use-case diagram of the marketplace

In addition to these functional requirements, several non-functional requirements guided the design: security (hashed passwords, protection against SQL injection and controlled file uploads), reliability (prevention of double bookings and consistent state transitions), usability (a responsive, animated and intuitive interface) and portability (single-command deployment through Docker).

2.3 Database Design

The persistent state of the platform is stored in a normalized MySQL relational schema consisting of thirteen tables. The design places every user - client, provider or administrator in a single users table distinguished by a role attribute, and extends providers with a one-to-one professional_profiles record. Services belong to a professional and a category; bookings reference a client, a service and a professional and carry a status and payment_status; reviews are linked one-to-one to a completed booking so that only genuine customers can rate a provider. Referential integrity is enforced through foreign keys, uniqueness through unique keys on e-mail and category slugs, and query performance through indexes on frequently filtered columns. The complete entity-relationship diagram is shown in Figure 2.3, and the role of each core entity is summarized in Table 2.2.

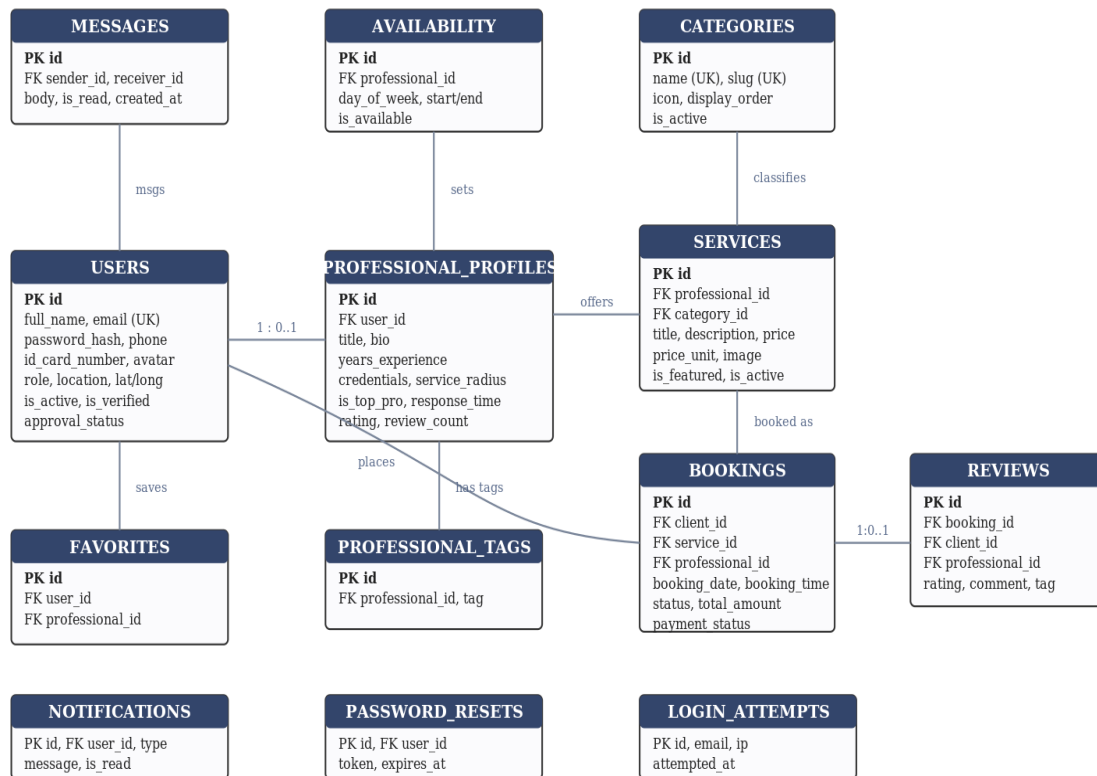


Fig. 2.3: Entity-relationship (ER) diagram of the database

Table 2.3: Core database entities and their purpose

Entity	Purpose
users	Accounts for clients, providers and admins; role, approval status, contact and location data.
professional_profiles	Extended provider information: title, bio, experience, rating and review count.
categories	Service categories (e.g., Plumbing, Cleaning) used for classification and filtering.
services	Individual services offered by a provider, with price, price unit and image.
bookings	A client's booking of a service, carrying date, time, status, amount and payment status.
reviews	One-to-one review of a completed booking; drives the provider's aggregate rating.
availability	Per-day working hours of a provider that generate the client-facing booking slots.
favorites, messages, notifications	Saved providers, in-app chat and event notifications respectively.
professional_tags	Free-form skill tags attached to a provider profile for search.
login_attempts, password_resets	Support tables for login rate-limiting and password recovery.

2.4 Technology Stack

The platform was built entirely with modern, widely-adopted and open-source technologies, chosen to reflect current industry practice and to keep the system portable and cost-free to operate. Table 2.1 lists the technologies used at each layer of the stack.

Table 2.4: Technology stack of the Neighbour platform

Layer	Technologies
Frontend	React 19, TypeScript, Vite 7, Tailwind CSS 3, React Router 7, GSAP, Lenis, lucide-react

Layer	Technologies
Backend	PHP 8.2 with Apache, PDO with prepared statements, session-based authentication
Database	MySQL 8 (InnoDB, foreign keys and indexes)
Security	bcrypt password hashing, role-based access control, login rate-limiting, validated uploads
Tooling / DevOps	Docker and Docker Compose, Git version control

2.5 REST API and Security Design

The backend exposes a resource-oriented REST API in which each endpoint corresponds to a domain resource and standard HTTP verbs express CRUD operations. Every write operation is protected on the server by role-based access control, so that, for example, only the owning provider may modify a service and only an administrator may approve a provider - client-side checks are treated purely as user-experience hints and never as security boundaries. Table 2.3 lists the principal endpoints.

Table 2.5: Principal REST API endpoints

Endpoint	Responsibility
auth	Registration, login, logout and session validation.
users	Account management and admin activation / deactivation.
professionals	Provider directory, profiles and admin approval.
services	Service catalogue with search, filter and CRUD.
bookings	Creation and lifecycle transitions of bookings.
reviews	Submission and retrieval of reviews for completed jobs.
availability	Per-provider weekly availability that drives booking slots.
favorites, messages, notifications	Saved providers, in-app chat and notifications.
stats	Aggregated sales, revenue and growth statistics for the admin.
upload	Validated image upload for services and profiles.

Security was treated as a first-class concern throughout the training. Passwords are stored only as bcrypt hashes and never in plain text. Every database query is executed through a PDO prepared statement, which eliminates the SQL-injection attack surface. Repeated failed logins are rate-limited using a dedicated login_attempts table to mitigate brute-force attacks. Uploaded images are validated by type and size before being stored and all sensitive operations require an authenticated session with an appropriate role.

2.6 Core Algorithms and Business Logic

Two pieces of business logic are central to the correctness of the marketplace: preventing double bookings and keeping provider ratings accurate. When a client requests a booking, the server first regenerates the provider's valid slots for the chosen day from their weekly availability, then confirms inside a single transaction that no confirmed or in-progress booking already occupies the requested slot before it inserts the new booking. Algorithm 2.1 outlines this procedure.

Algorithm 2.1: Double-booking-safe slot booking

Input: clientId, serviceId, date, time

Output: created booking, or rejection

- 1) provider $\leftarrow$ service.professional
 - 2) slots $\leftarrow$ GenerateSlots(provider.availability, date)
 - 3) if time $\notin$ slots then reject “outside working hours”
 - 4) **begin transaction**
 - 5) existing $\leftarrow$ bookings where professional = provider
 - 6) and date = date and time = time
 - 7) and status $\in$ {confirmed, in_progress}
 - 8) if existing $\neq \emptyset$ then rollback; reject “slot taken”
 - 9) amount $\leftarrow$ service.price
 - 10) insert booking(client, service, provider, date, time, amount, status = pending)
 - 11) commit; notify provider
 - end**
-

The platform earns revenue by charging a fixed fifteen percent commission on every completed booking. If A_i denotes the total amount of the i -th completed booking, the platform revenue R over a period is given by Equation 2.1.

$$R = 0.15 \times \sum A_i \quad (2.1)$$

Whenever a client submits a new review for a completed booking, the provider's aggregate rating is recomputed as the arithmetic mean of all their visible review scores and the review count is updated. If r_i is the score of the i -th of n reviews, the updated average rating $\bar{r}$ is given by Equation 2.2 and the procedure is summarized in Algorithm 2.2.

$$\bar{r} = (1 / n) \times \sum r_i \quad (2.2)$$

Algorithm 2.2: Provider rating recomputation on new review

Input: bookingId, clientId, score, comment

- 1) if booking.status $\neq$ completed then reject
 - 2) if review exists for booking then reject “already reviewed”
 - 3) insert review(booking, client, provider, score, comment)
 - 4) $R \leftarrow$ all visible reviews of provider
 - 5) provider.rating $\leftarrow$ mean(scores in R)
 - 6) provider.review_count $\leftarrow |R|$
 - 7) save provider; notify provider
-

CHAPTER III

Results and Discussion

This chapter presents the results of the industrial training: the fully functional marketplace that was implemented, the testing carried out to validate it, the security and performance characteristics achieved and a discussion of the outcomes and lessons learned.

3.1 Implemented System and Modules

The training produced a complete, working three-tier marketplace in which all planned modules were implemented and integrated. A client can register, browse services and a filterable professionals directory, open a rich profile, book an available slot, track bookings, review completed jobs, save favorites and chat with providers. A provider, once approved, manages services and weekly availability and progresses bookings through their lifecycle from a statistics dashboard. An administrator monitors live sales and platform revenue, approves providers, manages users and exports CSV reports. Table 3.1 summarizes the implemented modules by role.

Table 3.1: Implemented modules by user role

Role	Implemented modules
Client	Authentication, service and professional discovery with search / filter / sort, profile and review viewing, booking with slot selection, my-bookings and cancellation, reviews, favourites, messaging.
Service Provider	ID-verified registration with admin approval, statistics dashboard, service management with image upload, weekly availability, booking management (confirm / in-progress / complete / cancel), profile editing, chat and notifications.
Administrator	Live statistics, total sales and 15% platform revenue, provider approval / rejection, user activation, announcement banner, CSV export, growth and revenue charts.

Role	Implemented modules
Platform	Server-side RBAC, in-app notifications, login rate-limiting, prepared statements, validated uploads, responsive UI and offline demo fallback.

A booking moves through a well-defined lifecycle that keeps the state of the system consistent for both client and provider. A newly created booking is pending; the provider confirms it, marks it in progress and finally completes it, at which point the platform commission is realized and the client becomes eligible to leave a review. Either party may cancel a booking before completion. These transitions are shown in Figure 3.1.

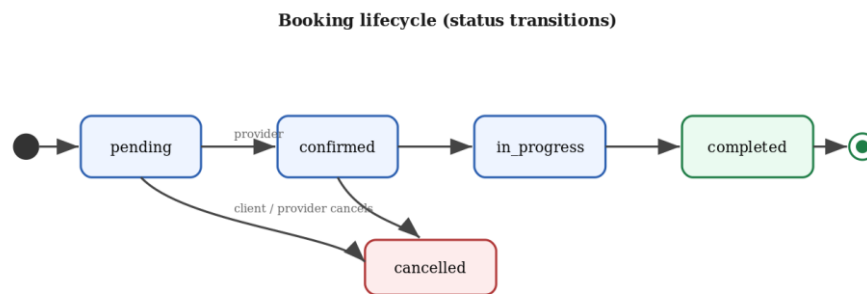


Fig. 3.1: Booking lifecycle (status transitions)

To illustrate the working system, representative screens from the client, service-provider and administrator interfaces are presented in Figures 3.2–3.10

Client interface.

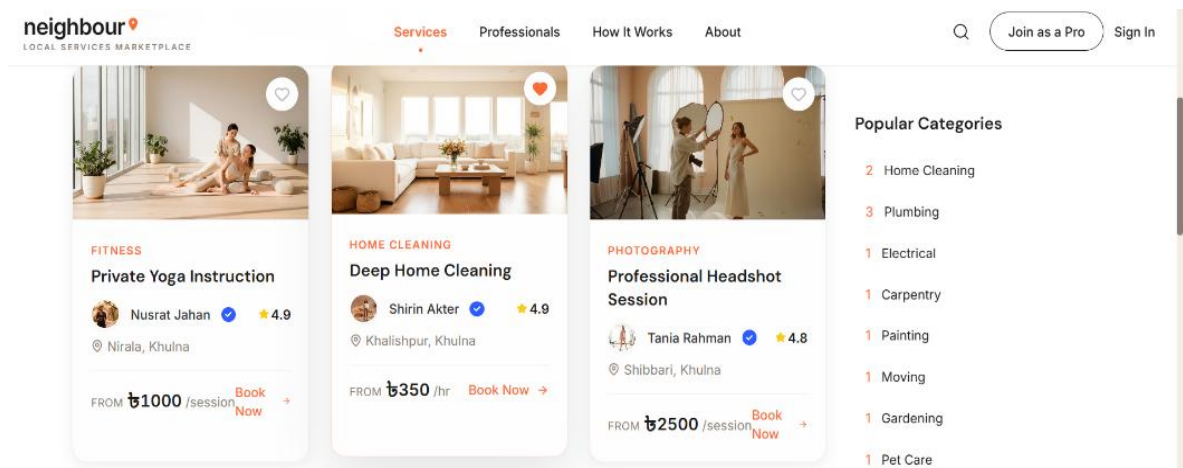


Fig. 3.2: Client-side services directory (search, filter and sort)

This page presents every available service as a card showing the service name, category, provider, rating, location and starting price. Clients can search by keyword, narrow the results using the category filters on the right and sort them, then press “Book Now” to start a booking.

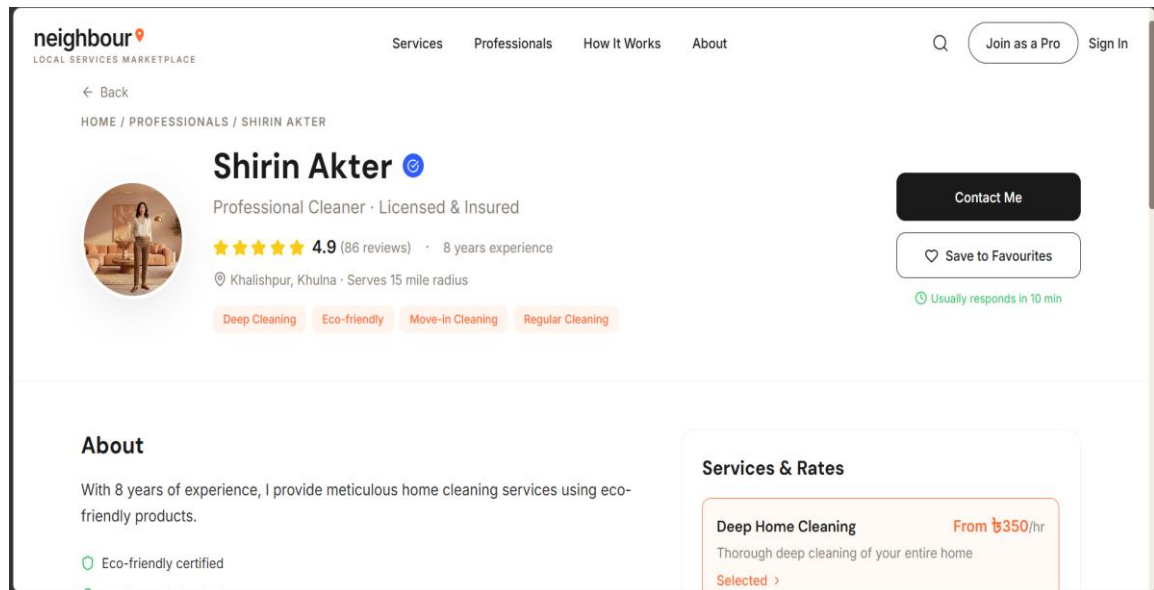


Fig. 3.3: Professional profile page with ratings and service rates

A provider's public profile displays their photo, verification badge, average rating, years of experience, service area and skill tags, followed by an "About" section and a "Services & Rates" list. From here a client can contact the provider, save them to favourites and choose a specific service to book.

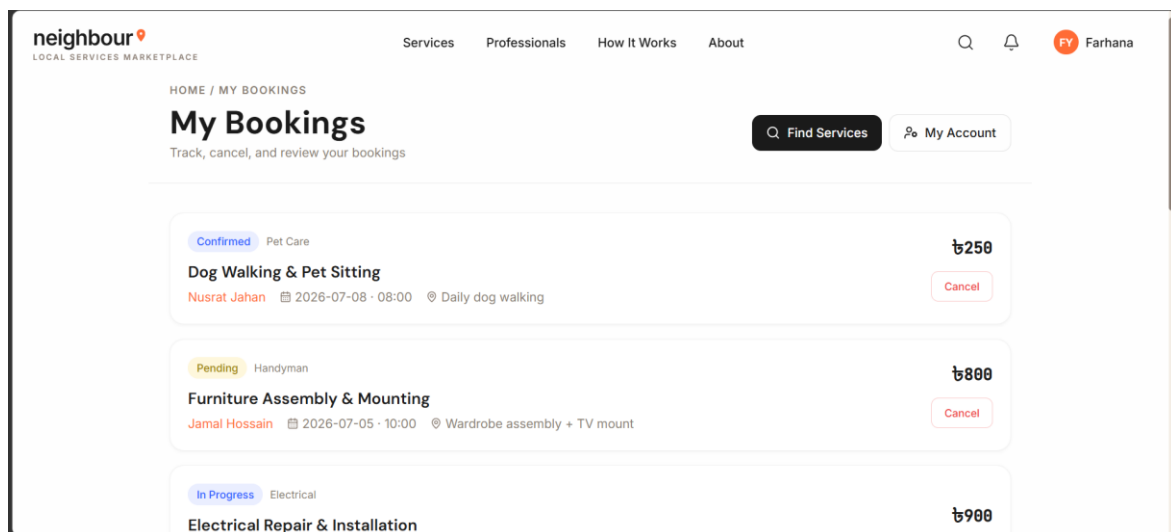


Fig. 3.4: Client My Bookings page (tracking and cancellation)

The My Bookings page lets a client track every booking, each tagged with its current status (Confirmed, Pending or In Progress) along with the service, professional, date, time, note and amount. A booking can be cancelled from this screen at any time before it is completed.

Service-provider interface.

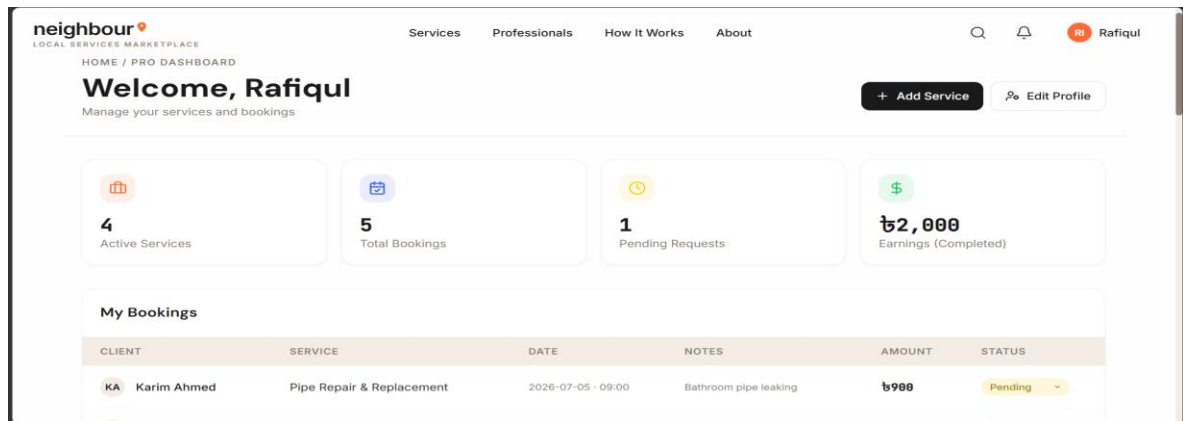


Fig. 3.5: Provider statistics dashboard

After signing in, a provider lands on a personal dashboard that summarises active services, total bookings, pending requests and completed earnings, with a table of incoming bookings below. It gives the provider an at-a-glance view of their workload and income.

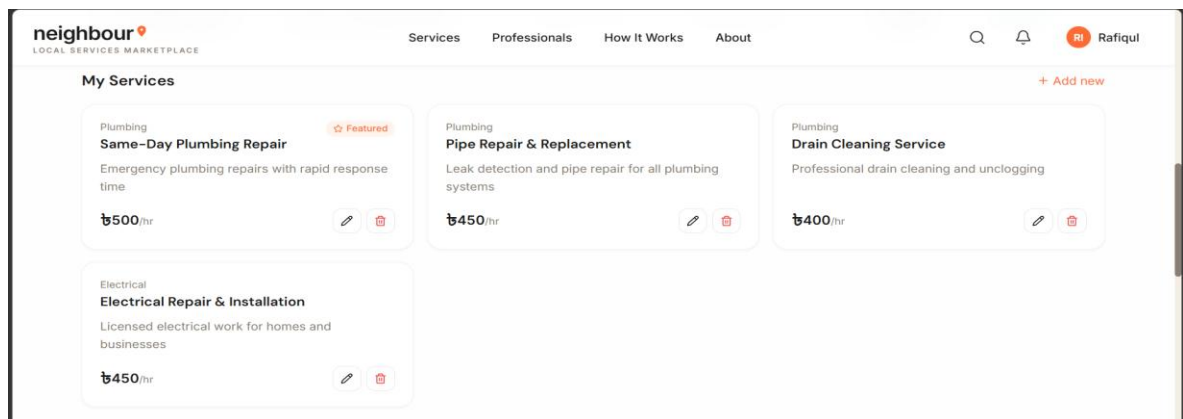


Fig. 3.6: Provider service management screen

On the service-management screen a provider maintains their catalogue of services; each card shows the category, title, description and hourly rate with edit and delete controls, and a “Featured” tag marks services promoted on the homepage. The “+ Add new” button opens the form used to create additional services.

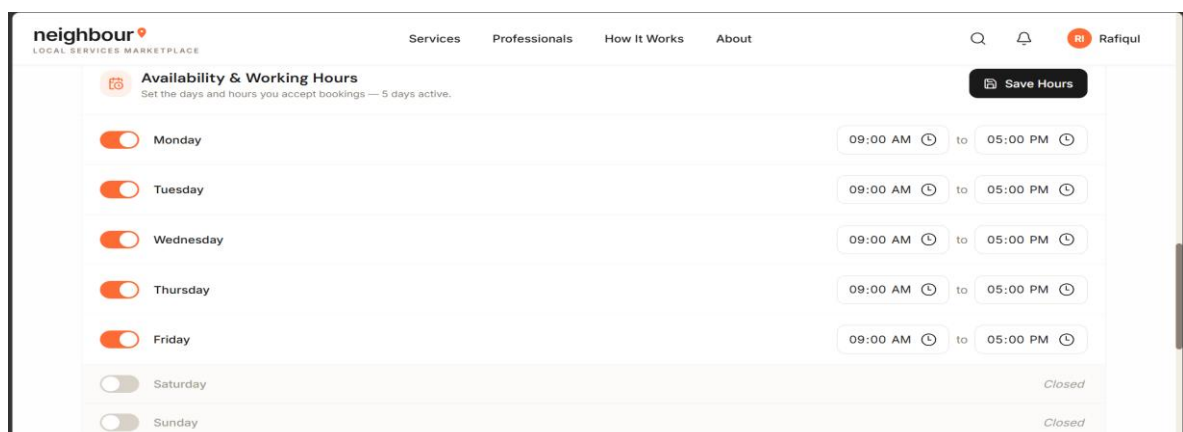


Fig. 3.7: Provider weekly availability configuration

Here a provider configures weekly availability by switching each day on or off and setting its start and end times. These active hours automatically generate the bookable time slots offered to clients, while days left switched off are shown as “Closed”.

Administrator interface.

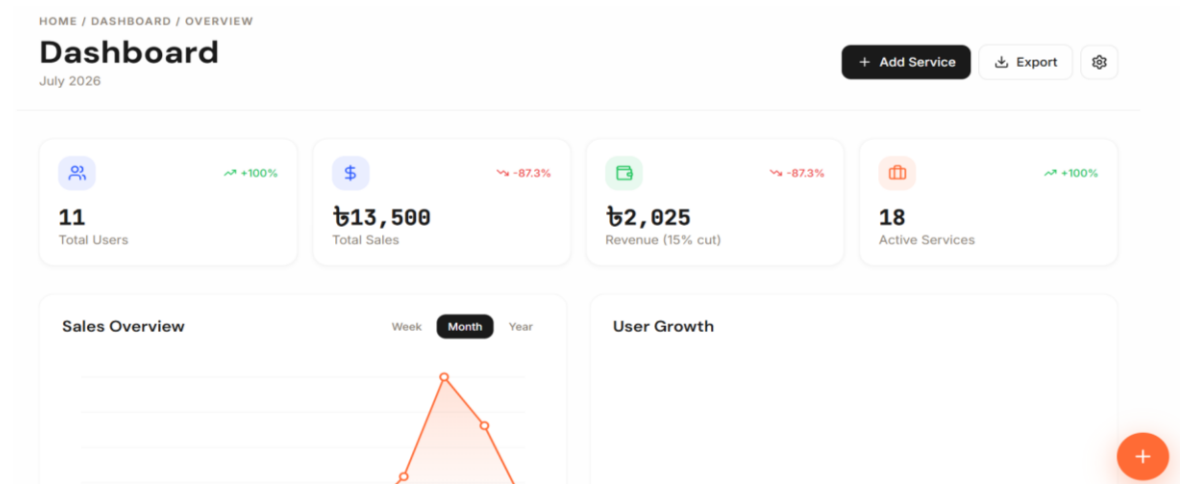


Fig. 3.8: Administrator dashboard (sales, revenue and growth)

The administrator dashboard reports live platform metrics - total users, total sales, the platform’s 15% revenue share and the number of active services alongside Sales Overview and User Growth charts that can be viewed by week, month or year.

USER	ROLE	LOCATION	JOINED	STATUS	ACTION
IK Imran Kabir imran.kabir@gmail.com	Professional AC & Refrigeration Technician ID: 1990-1122334455	Boyra, Khulna	2025-12-16	Pending	Deactivate
AU Admin User admin@neighbour.com	Admin	Khulna, Bangladesh	2026-07-26	Active	You
RI Rafiqul Islam rafiquel.islam@gmail.com	Professional Master Plumber	Sonadanga, Khulna	2026-05-09	Active	Deactivate
SA Shirin Akter shirin.akter@gmail.com	Professional Professional Cleaner	Khalishpur, Khulna	2026-04-15	Active	Deactivate
TR Tania Rahman tania.rahman@gmail.com	Professional Portrait Photographer	Shibbari, Khulna	2026-03-22	Active	Deactivate
JH Jamal Hossain jamal.hossain@gmail.com	Professional Handyman	Daulatpur, Khulna	2026-02-26	Active	Deactivate

Fig. 3.9: Administrator user and provider management

The user-management table lists every account with its name, email, role, professional title, location, join date and status. From the Action column the administrator can approve a pending provider or activate and deactivate any account.

Messages

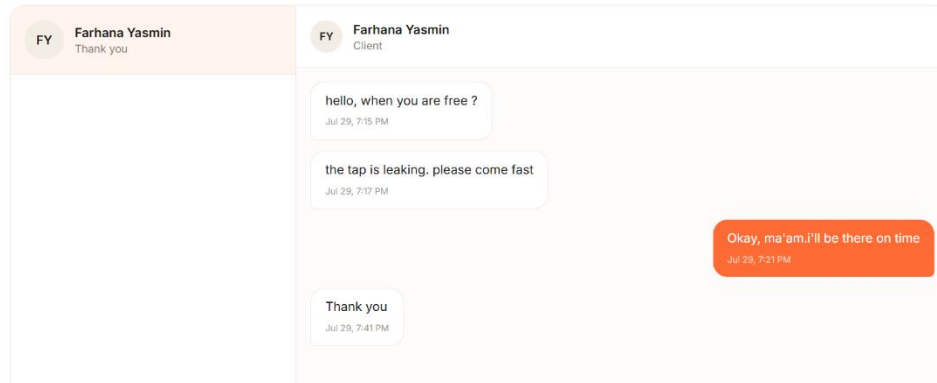


Figure 3.10: Clients (User) and Service provider in app messaging

The messaging module enables direct communication between clients and service professionals within the platform. The conversation list on the left displays all active chat threads, each showing the contact's name and a preview of the most recent message. The selected thread on the right shows the full exchange between the professional (Rafiqul) and client Farhana Yasmin, including message content and individual timestamps for each reply

3.2 Testing and Validation

The system was validated through functional testing of each module against its requirements, with particular attention to the critical paths of authentication, booking and administrative approval. Representative test cases and their outcomes are listed in Table 3.2. All listed cases passed, confirming that the core business rules - role enforcement, double-booking prevention, review eligibility and revenue calculation behave as specified.

Table 3.2: Representative test cases and outcomes

No.	Test case	Expected result	Outcome
T1	Register and log in as a client with valid credentials	Session created, client dashboard shown	Pass
T2	Provider signs up with National ID	Account created as pending approval	Pass
T3	Book a slot already taken for the same provider	Booking rejected as slot taken	Pass

No.	Test case	Expected result	Outcome
T4	Leave a review for a non-completed booking	Review rejected	Pass
T5	Client attempts an admin-only action	Request denied by server (RBAC)	Pass
T6	Admin approves a pending provider	Provider becomes visible and can log in	Pass
T7	Complete a booking and check admin revenue	Sales and 15% revenue update correctly	Pass
T8	Repeated failed logins from one client	Further attempts rate-limited	Pass

3.3 Security and Performance

From a security standpoint, the platform meets the objectives set for the training. Credentials are protected by bcrypt hashing; the exclusive use of PDO prepared statements removes the SQL-injection surface; login rate-limiting mitigates brute-force attacks; uploads are validated; and every state-changing request is authorized on the server by role. In terms of performance, indexing of frequently queried columns keeps directory search, filtering and statistics queries responsive, while the React single-page frontend, served as a static bundle, gives fast client-side navigation. The offline demo fallback further improves perceived reliability by keeping the interface usable even when the backend is temporarily unavailable.

3.4 Discussion

The industrial training successfully delivered a realistic, end-to-end product rather than an isolated exercise and in doing so exercised the full software-development lifecycle: requirement analysis, database and API design, frontend and backend implementation, containerized deployment and testing. The most instructive challenges were architectural rather than syntactic. Guaranteeing that two clients could never book the same slot required moving the check inside a database transaction; keeping provider ratings trustworthy required binding each review to a single completed booking; and keeping the system secure required consistently treating the server, not the browser, as the authority for every

decision. Containerizing the stack with Docker eliminated “works-on-my-machine” problems and reflected how software is delivered in industry. Collectively, these experiences translated classroom concepts into concrete professional engineering practice.

CHAPTER IV

Conclusions and Recommendations

4.1 Conclusions

This report has presented the design and development of Neighbour, a full-stack web-based local services marketplace built during the CSE 4100 Field Work / Industrial Training. The platform connects the residents of Khulna with verified local professionals and supports the complete journey of discovery, booking, service delivery, review and payment settlement through a fair 15% commission model. It was implemented as a portable three-tier system using React and TypeScript on the frontend, a secure PHP 8.2 REST API on the backend and a normalized MySQL database, all orchestrated with Docker. All the objectives defined at the outset of the training - requirement analysis, database and API design, secure implementation of the three user roles, correct business logic and containerized deployment - were met and validated through testing. Beyond the artefact itself, the training provided invaluable exposure to professional software-engineering practices and consolidated the author's understanding of full-stack web development.

4.2 Limitations

Despite achieving its objectives, the current system has some limitations. Payments are modelled and settled logically but are not yet charged through a real payment gateway, so money does not actually move within the platform. Notifications are delivered in-app only, without e-mail or SMS delivery. Search is attribute-based and does not yet exploit the stored geographic coordinates for true proximity ranking. Finally, the platform is a web application only and does not yet offer a dedicated native mobile client, which would suit many of the target users.

4.3 Recommendations and Future Work

Several enhancements are recommended to evolve Neighbour from a training project into a production-ready service. First, a real online payment gateway such as bKash,

SSLCommerz or Stripe should be integrated so that commissions and provider payouts are handled automatically and securely. Second, e-mail and SMS notification channels should be added alongside the existing in-app notifications to improve engagement. Third, geolocation-based search and ranking should be implemented using the latitude and longitude already stored in the schema, so that clients are matched with the nearest suitable professionals. Fourth, a native mobile application would broaden accessibility for the target audience. Longer-term improvements include automated testing and continuous integration pipelines, richer analytics for administrators and providers, and a recommendation system that suggests professionals based on a client's history and preferences.

References

- [1] R. T. Fielding and R. N. Taylor, “Principled design of the modern Web architecture,” *ACM Transactions on Internet Technology*, vol. 2, no. 2, pp. 115–150, 2002, doi: 10.1145/514183.514185.
- [2] Meta Open Source, “React – The library for web and native user interfaces,” 2024. Available: <https://react.dev/>
- [3] Microsoft, “TypeScript: JavaScript with syntax for types,” 2024. Available: <https://www.typescriptlang.org/>
- [4] The PHP Group, “PHP 8.2 Manual,” 2024. Available: <https://www.php.net/manual/en/>
- [5] Oracle Corporation, “MySQL 8.0 Reference Manual,” 2024. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- [6] D. Merkel, “Docker: Lightweight Linux containers for consistent development and deployment,” *Linux Journal*, vol. 2014, no. 239, art. 2, 2014.
- [7] N. Provos and D. Mazières, “A future-adaptable password scheme,” in *Proc. USENIX Annual Technical Conference*, 1999, pp. 81–91.
- [8] OWASP Foundation, “OWASP Top 10 – 2021: The ten most critical web application security risks,” 2021. Available: <https://owasp.org/Top10/>